Design of Custom Instructions on PicoRV32 Processor for Accelerating Fast Fourier Transform Algorithm

1st Abidul Qohar
Department of Electrical Engineering and Information Technology
Faculty of Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
abidulqohar@mail.ugm.ac.id

2nd Agus Bejo
Department of Electrical Engineering and Information Technology
Faculty of Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
agusbj@ugm.ac.id

3rd Eka Firmansyah
Department of Electrical Engineering and Information Technology
Faculty of Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
eka.firmansyah@ugm.ac.id

***Abstract*—Audio equalizer applications rely on the Fast Fourier Transform (FFT) to transform time-domain signals into the frequency domain for selective gain adjustment. On resource-constrained embedded processors implementing only the base RV32I integer instruction set, FFT computation becomes a performance bottleneck due to the absence of hardware multiplication support and the repetitive nature of the butterfly operation. This paper presents the implementation of custom RISC-V instructions to accelerate the butterfly computation of a radix-2 FFT integrated into a PicoRV32-based audio equalizer system. Four custom instructions—mshift, bload, bfly, and bget are designed to collapse the four input, two output butterfly operation into a single hardware-accelerated sequence and minimize the multiplication operations through the PicoRV32 Custom Peripheral Interface (PCPI). The GNU assembler is modified to recognize the new instructions, enabling their use through inline assembly in C code without requiring compiler backend changes. A complete System-on-Chip is implemented in Verilog comprising the PicoRV32 core, the FFT custom instruction module, ROM and dual RAM blocks, and AXI bus interconnect. Experimental results from Icarus Verilog simulation show that processing 1024 audio samples through a 64-point FFT, 3-band equalizer, and inverse FFT requires 5.84 million clock cycles with custom instructions enabled, compared to around 17.08 million cycles in the software-only baseline a speedup of 2.92×. Functional correctness is verified against a NumPy double-precision floating-point reference, achieving a signal-to-noise ratio of 56.28 dB. These results demonstrate that profile-driven custom instruction design is an effective strategy for accelerating signal processing workloads on minimal RISC-V cores.**

***Index Terms*—RISC-V, custom instruction, Fast Fourier Transform, audio equalizer, PicoRV32, hardware acceleration**

I. Introduction

Audio signal processing on embedded systems has become increasingly important for applications such as smart speakers, hearing aids, in-vehicle audio systems, and wearable devices [1]. A common operation in these systems is the audio equalizer, which selectively amplifies or attenuates specific frequency bands of an audio signal. Modern equalizer implementations typically transform the signal into the frequency domain using the Fast Fourier Transform (FFT), apply per-band gain coefficients, and reconstruct the time-domain output via the inverse FFT. The FFT, while asymptotically efficient at $O(N \log N)$, imposes significant computational cost on embedded processors. The algorithm consists of two principal stages: bit reversal permutation of the input indices, and $\log_2(N)$ stages of butterfly operations, each involving complex-number multiplication and addition [2]. When executed on a general-purpose processor implementing only the base RV32I instruction set, which lacks hardware integer multiplication, each butterfly multiplication must be emulated through sequences of shift and add instructions, dramatically inflating the cycle count. RISC-V, an open and royalty-free instruction set architecture, addresses this limitation through its modular design philosophy. The specification reserves opcode space for user-defined custom instructions, allowing designers to extend the base ISA with application-specific operations without modifying the standard [4]. This extensibility makes RISC-V particularly suitable for domain-specific acceleration of computational bottlenecks. Prior work has explored instruction set extensions to accelerate FFT and related digital signal processing workloads. One line of research integrates dedicated FFT coprocessors with RISC-V cores through specialized coupling interfaces, in which a large block of the transform is offloaded to custom hardware [5], [6]. Such designs deliver substantial acceleration but rely on heavyweight coprocessor interfaces and incur significant area overhead, which is difficult to justify on minimal embedded cores. A second line of work targets DSP workloads more broadly, extending the RISC-V ISA for audio denoising [7] and ultra-low-power wireless signal processing [8], while custom instructions for FFT on non-RISC-V soft cores have also been investigated [**?**]. These approaches, however, are generally evaluated on processors that already provide hardware integer multiplication through the standard M extension, so the cost of emulating multiplication in software is not part of the problem they address.

Few studies have considered the combined effect of two

constraints that characterize minimal RISC-V cores such as PicoRV32: the absence of the M extension, which forces every multiplication to be emulated in software, and the limited two-source operand bandwidth of a lightweight custom-instruction interface, which is insufficient for the four-operand butterfly. The specific access pattern of FFT-based audio equalization on such a core, where both of these constraints act simultaneously, has not been directly addressed. This paper targets that gap. It presents a profile-driven custom instruction design that resolves both constraints on an unmodified PicoRV32 core, using only the standard PicoRV32 Custom Peripheral Interface (PCPI) and without introducing a separate coprocessor or modifying the core pipeline.

II. Background

A. Fast Fourier Transform and the Butterfly Operation

The Discrete Fourier Transform (DFT) of an N-point complex sequence $x[n]$ is defined as [3] equation 1.

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot W_N^{nk}, \quad W_N = e^{-j2\pi/N} \tag{1}$$

Direct computation requires $O(N^2)$ complex multiplications. The radix-2 decimation-in-time FFT reduces this to $O(N \log_2 N)$ by recursively decomposing the DFT into smaller transforms, exploiting the periodicity and symmetry of the twiddle factor W_N [2], [3]. The algorithm comprises two stages. The first stage is bit reversal, reorders the input sequence by reversing the binary representation of each index. For $N = 8$, for example, index 1 (binary 001) is swapped with index 4 (binary 100), and index 3 (011) is swapped with index 6 (110).

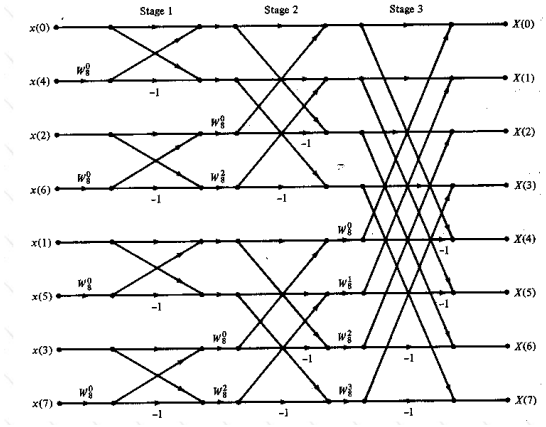

Fig. 1. Butterfly unit process with 8 input data points to obtain frequency domain output [15]

The second stage shown in Fig. 1 performs $\log_2(N)$ passes of butterfly operations. Each butterfly combines two complex inputs a and b using a twiddle factor W_N^r to produce two outputs, requiring four real multiplications and two real additions [3] :

$$A = a + W_N^r \cdot b \tag{2}$$

$$B = a - W_N^r \cdot b \tag{3}$$

Expanding the complex multiplication $W_N^r \cdot b$ where $W_N^r = c - js$ ($c = \cos$, $s = \sin$) and $b = b_r + jb_i$:

$$\text{tr} = c \cdot b_r + s \cdot b_i \tag{4}$$

$$\text{ti} = c \cdot b_i - s \cdot b_r \tag{5}$$

A single butterfly therefore requires four real multiplications and two real additions/subtractions. For an N-point FFT, the total number of butterfly operations is $(N/2) \cdot \log_2(N)$. For $N = 64$, this yields 192 butterflies, each requiring four multiplications totaling 768 multiplications per FFT.

B. PicoRV32 Architecture

PicoRV32 is a size-optimized RISC-V soft-core processor that implements the base RV32I integer instruction set, with the RV32IM and RV32IMC extensions available as optional configuration parameters [9]. The core uses a sequential, non-pipelined execution model in which each instruction completes before the next is fetched; this simplicity both reduces area and simplifies the integration of custom hardware, since there is no pipeline hazard logic to disturb.

Two architectural properties of the configuration used in this work directly shape the design problem addressed by this paper. First, the core is configured as a pure RV32I implementation, without the `M` extension, so it provides no hardware integer multiplier. Every multiplication in compiled code is therefore expanded by the compiler into a sequence of shift and add instructions, an emulation cost that is incurred on every multiply and is quantified in Section III.

Second, the core does not implement a hardware Floating-Point Unit (FPU) and does not provide the RISC-V standard `F` extension, again to preserve a minimal area footprint. As a consequence, any use of IEEE-754 floating-point arithmetic would force the compiler to rely on software emulation (soft-float), in which each floating-point operation expands into a long sequence of integer instructions. For the latency-critical inner loops of a real-time audio equalizer, this overhead is prohibitive. The absence of an FPU is therefore the architectural reason that this work adopts a fixed-point numerical format, described in Section II-D.

C. The PCPI Interface

To allow the instruction set to be extended without modifying the core itself, PicoRV32 provides the PicoRV32 Custom Peripheral Interface (PCPI) as in Listing 1.

```
1  // Pico Co-Processor Interface (PCPI)
2  output reg        pcpi_valid,
3  output reg [31:0] pcpi_insn,
4  output     [31:0] pcpi_rs1,
5  output     [31:0] pcpi_rs2,
6  input             pcpi_wr,
7  input      [31:0] pcpi_rd,
8  input             pcpi_wait,
9  input             pcpi_ready,
```

Listing 1. PCPI Signals Interface in PicoRV32

When the core fetches an instruction whose opcode is not part of the implemented base ISA, it does not raise an illegal-instruction trap. Instead, it forwards the complete 32-bit instruction word together with the two source register values (`rs1` and `rs2`) onto the PCPI bus. An attached PCPI module inspects the instruction, decodes its `funct3` and `funct7` fields to select the requested operation, performs the computation, and signals completion by asserting `pcpi_ready`. A result value may optionally be returned to the core through the `pcpi_rd` signal, which the core writes back to the destination register. During this exchange the core enters a stall state and waits until `pcpi_ready` is asserted, so the custom operation may take a single cycle or multiple cycles without any change to the core's control logic.

Because this mechanism leaves the PicoRV32 pipeline, register file, and decoder untouched, a custom instruction module can be attached purely as an external block. It allows the proposed FFT instructions to be added and verified without forking or re-validating the core itself.

D. Fixed-Point Arithmetic

Because the PicoRV32 core used in this work provides no hardware FPU (Section II-B), all FFT computations are carried out in fixed-point arithmetic, which maps directly onto the available RV32I integer datapath. In a fixed-point representation, a real number is stored in an ordinary integer register, and the binary point is fixed at a position agreed upon by convention rather than encoded in the data. A $Qm.f$ format denotes a signed value with m integer bits and f fractional bits, so the stored integer corresponds to the real value scaled by 2^f. Arithmetic on these values then reduces to ordinary integer arithmetic on the scaled representation, which is exactly what the RV32I instruction set provides. Sample data uses a Q21.10 representation: a signed 32-bit integer with 1 sign bit, 21 integer bits, and 10 fractional bits.

III. Custom Instruction Design

A. Profiling and Bottleneck Identification

To establish a baseline for acceleration, the audio equalizer application was first executed on a standard RV32I architecture without custom extensions. An instruction-level profiling trace [16] was generated to analyze the execution behavior of the Fast Fourier Transform (FFT) during the real-time audio buffering loop.

Table I presents the ten most frequent instruction pairs observed in the compiled FFT routine. The profiling reveals that shift-and-add patterns dominate the instruction stream, with `srli–slli`, `andi–beqz`, and `slli–andi` together accounting for over 840,000 dynamic instructions. These patterns arise from two sources: bit reversal index manipulation (the `andi–beqz` and shift sequences for index bit operations) and software emulation of multiplication in the butterfly computation (the `slli–add` chains, which implement shift-and-add multiplication in the absence of a hardware multiplier).

TABLE I
Top 10 looping Instruction Pairs in the Compiled Audio Equalizer (RV32I)

Rank	Instruction Pair	Count
1	`srli` → `slli`	291,707
2	`andi` → `beqz`	289,144
3	`slli` → `andi`	260,037
4	`lw` → `slli`	94,403
5	`add` → `lw`	62,111
6	`lw` → `sub`	29,228
7	`addi` → `ble`	29,124
8	`slli` → `mv`	28,641
9	`mv` → `andi`	28,532
10	`sub` → `sw`	26,913

This instruction distribution exposes a severe bottleneck. The fundamental mathematical equation for the FFT Butterfly requires four simultaneous inputs (real data, imaginary data, and two trigonometric twiddle factors). Because the standard Pico Co-Processor Interface (PCPI) restricts data transfer to a maximum of two source registers (rs1 and rs2) per clock cycle, the standard GCC compiler is forced into an aggressive register-spilling routine. The CPU exhausts the majority of its clock cycles orchestrating data movement calculating array pointers, loading operands, and storing intermediate cross-multiplications back to the main memory leaving the Arithmetic Logic Unit (ALU) heavily underutilized.

B. Design Constraints

The RISC-V R-type instruction format encodes two source registers (`rs1`, `rs2`) and one destination register (`rd`) within a 32-bit instruction word. However, a single butterfly operation requires four input operands—the complex input pair (b_r, b_i) and the twiddle factor pair (c, s) and produces two output operands (tr, ti). This exceeds the operand capacity of a single R-type instruction.

C. Instruction Set Architecture

To accelerate the computational bottlenecks identified during profiling, the baseline RISC-V Instruction Set Architecture (ISA) is extended with four custom instructions. Rather than relying on a single opcode, the extensions strategically utilize two reserved spaces within the RISC-V custom opcode map: `0x0B` (`custom-0`) and `0x2B` (`custom-1`). The instructions are uniquely decoded using a combination of their `funct3` and `funct7` fields, as detailed in Table II.

TABLE II
Custom Instruction Encoding

Instruction	funct7	rs2	rs1	funct3	rd	opcode
`mshift`	0x00	in a	in b	0x0	out c	0x0B
`bload`	0x00	imag	real	0x0	x	0x2B
`bfly`	0x01	s	c	0x1	tr	0x2B
`bget`	0x02	x	x	0x2	ti	0x2B

The semantics of each instruction are:

- `mshift rd, rs1, rs2`: Reads the input a from `rs1` and b from `rs2` then computes both for multiplication 32-bit in 1 cycle and writes the output c 32-bit into register destination (`rd`).
- `bload rs1, rs2`: Stores the values of `rs1` (b_r, real component of input) and `rs2` (b_i, imaginary component) into internal hardware registers within the PCPI module. No result is written to the register file.
- `bfly rd, rs1, rs2`: Reads twiddle factor c from `rs1` and s from `rs2`, computes both $tr = (c \cdot b_r + s \cdot b_i) \gg 10$ and $ti = (c \cdot b_i - s \cdot b_r) \gg 10$ using the previously loaded inputs, latches ti internally, and writes tr to `rd`.
- `bget rd`: Returns the latched ti value to `rd`.

D. Hardware Implementation

The PCPI FFT module is instantiated alongside the PicoRV32 core and connected via the standard PCPI bus signals. Fig. 2 shows the block diagram of the integrated system.

Upon receiving a PCPI request, the FFT module decodes the instruction's `funct3` and `funct7` fields. For `bload`, the module captures `rs1` and `rs2` into internal `b_real` and `b_imag` registers and asserts `pcpi_ready` in the next cycle. For `bfly`, the module performs four 16×32-bit multiplications and the required addition/subtraction in combinational logic, applying the Q21.10 normalization shift, latches the ti result, and returns tr via `pcpi_rd` while asserting `pcpi_ready`. For `bget`, the module immediately returns the latched ti value.

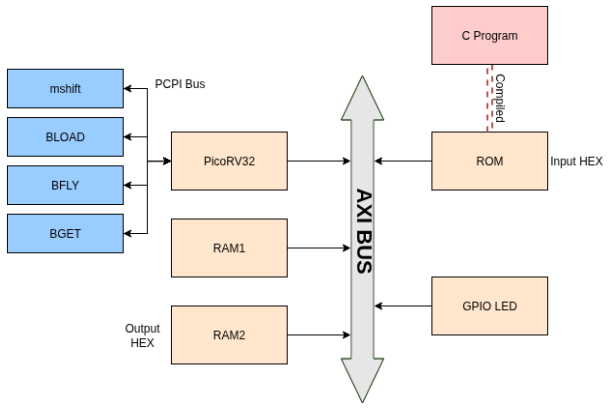

Fig. 2. Block diagram of the PicoRV32-based audio equalizer SoC with PCPI FFT custom instruction module.

The complete SoC is implemented in Verilog and simulated using Icarus Verilog. The system architecture comprises:

- PicoRV32 core: 32-bit RV32I implementation with PCPI enabled.
- PCPI FFT module: implements `bload`, `bfly`, `bget` as described in Section IV.
- ROM: 32 KB instruction memory loaded with the compiled audio equalizer program.
- Input RAM: stores the 1024-sample test audio waveform.
- Output RAM: 4 KB block for capturing the processed audio output stream (accessed via `MAGIC_ADDR 0x00020000`).
- AXI bus: provides addressable interconnection between the core, memories, and GPIO peripherals.

Among the four custom instructions, `bfly` carries the arithmetic core of the butterfly and is therefore the most representative of the hardware design. The PCPI module decodes it when the instruction word carries opcode `0x2B` and `funct3 = 1`, interpreting `rs1` as the cosine twiddle term c and `rs2` as the sine term s. These are combined with the real and imaginary operands `s_real_64` and `s_imag_64` that were captured by a preceding `bload`. Listing 2 shows the relevant portion of the Verilog implementation.

```
wire signed [63:0] r_c = s_real_64 * c_64;
wire signed [63:0] i_s = s_imag_64 * s_64;
wire signed [63:0] r_s = s_real_64 * s_64;
wire signed [63:0] i_c = s_imag_64 * c_64;

wire signed [63:0] ti_64 = r_s + i_c;

wire signed [31:0] tr_result = tr_64[FIXED_SHIFT
    +: 32];
wire signed [31:0] ti_result = ti_64[FIXED_SHIFT
    +: 32];

else if (is_bfly) begin
cached_ti  <= ti_result;
pcpi_rd    <= tr_result;
pcpi_wr    <= 1'b1;
pcpi_ready <= 1'b1;
end
```

Listing 2. Verilog implementation of the `bfly` custom instruction.

A single `bfly` instruction serves both the forward and inverse transform stages of the equalizer pipeline, with no dedicated inverse-FFT hardware. This reuse follows directly from the relationship between the forward and inverse DFT kernels. The forward transform applies the twiddle factor $W_N^r = e^{-j2\pi r/N} = c - js$, whereas the inverse transform applies its complex conjugate $W_N^{-r} = e^{+j2\pi r/N} = c + js$, where $c = \cos(2\pi r/N)$ and $s = \sin(2\pi r/N)$. The cosine term c is identical for both directions; only the sign of the sine term s differs.

The PCPI output signals `pcpi_rd`, `pcpi_wr`, and `pcpi_ready` are registered rather than driven combinationally. A combinational `pcpi_ready` can glitch when the decode signals settle within a cycle, and because the PicoRV32 core samples `pcpi_ready` at the clock edge such a glitch could cause the instruction to be acknowledged incorrectly. Registering the outputs guarantees a clean, stable handshake at every clock edge, at the cost of one additional cycle of latency: an instruction presented on `pcpi_valid` in cycle N produces its result and asserts `pcpi_ready` in cycle $N+1$.

E. Software Integration

To seamlessly bridge the high-level audio equalizer application with the underlying custom DSP hardware, the proposed instructions are exposed to the C programming environment via inline assembly macros. This methodology

provides explicit, cycle-accurate control over the coprocessor invocation without requiring extensive modifications to the GCC compiler's backend instruction scheduler.

```
#define hw_bload(real_val, imag_val) \
  __asm__ volatile ("bload %0, %1" :: \
    "r"((int32_t)(real_val)), \
    "r"((int32_t)(imag_val)) : "memory")

#define hw_bfly(tr_out, c_val, s_val) \
  __asm__ volatile ("bfly %0, %1, %2" \
    : "=r"(tr_out) \
    : "r"((int32_t)(c_val)), \
      "r"((int32_t)(s_val)) : "memory")

#define hw_bget(ti_out) \
  __asm__ volatile ("bget %0" \
    : "=r"(ti_out) :: "memory")
```

Listing 3. C macros for invoking the custom FFT instructions

To enable the assembler to recognize the new mnemonics, the `opcodes/riscv-opc.c` source file in GNU binutils was modified to add entries for `bload`, `bfly`, and `bget` with their corresponding encoding fields. The modified binutils is rebuilt and used in place of the standard toolchain when compiling FFT code.

IV. Results and Discussion

A. Compiler Synthesis and Instruction Validation

To verify the successful integration of the custom toolchain and the high-level C macros, the compiled binary was disassembled and analyzed using the RISC-V GNU objdump utility. The primary objective of this verification was to ensure that the GCC compiler respected the sequential integrity of the custom DSP operations without injecting redundant memory spills or reordering the instructions.

As shown in Fig. 3, the disassembled object code confirms the flawless translation of the C macros into the reserved `0x2B` custom opcode space. Crucially, the compiler successfully fetches the array values into registers `(a4, a5)` and immediately feeds them into the `bload` instruction. Protected by the inline assembly memory barriers, the subsequent `bfly` and `bget` instructions are emitted in perfect consecutive order.

```
236     380: 00072703           lw   a4,0(a4)
237     384: 00e7802b           bload    a5,a4
238     388: fca41783           lh   a5,-54(s0)
239     38c: fc841703           lh   a4,-56(s0)
240     390: 40e00733           neg  a4,a4
241     394: 00e797ab           bfly a5,a5,a4
242     398: fcf42e23           sw   a5,-36(s0)
243     39c: 000027ab           bget a5
244     3a0: fcf42c23           sw   a5,-40(s0)
245     3a4: fec42783           lw   a5,-20(s0)
```

Fig. 3. Assembly output generated from the compiler showing the correct custom instructions `bload`, `bfly`, and `bget` with proper register usage.

While necessary memory operations such as `sw` (store word) and `lw` (load word) are natively present to update the final array values, the disassembled code provides empirical proof that all redundant intermediate register spilling has been successfully eliminated. The `bload`, `bfly`, and `bget` sequence flawlessly executes the complex four-variable Butterfly computation without ever requiring the CPU to temporarily offload incomplete arithmetic states to the RAM. By effectively confining the intense mathematical cross-multiplications entirely within the coprocessor's hardware state, the system successfully bypasses the bottleneck and is fully optimized for the cycle-accurate performance evaluation detailed in the subsequent section.

B. Functional Correctness Verification

The hardware-accelerated FFT output was verified against a double-precision floating-point reference implemented in Python using NumPy's `fft` and `ifft` functions on identical input data. The Python reference applied the same 3-band equalizer gain pattern ($2.0\times$, $0.5\times$, $0.25\times$ in Q10) to ensure direct comparison.

Each 32-bit Q21.10 fixed-point output sample from the hardware simulation was converted to floating-point by dividing by $2^{10} = 1024$ prior to comparison. Error metrics over the 1024-sample output sequence are presented in Table III.

TABLE III
Correctness Verification Metrics

Metric	Value
Samples compared	1024
Maximum absolute error	4.94
Mean absolute error	0.92
RMS error	1.30
Mean relative error	0.28%
SNR (hardware vs. reference)	56.28 dB

The error metrics in Table III confirm that the fixed-point hardware implementation produces results consistent with the double-precision floating-point reference. The 56.28 dB signal-to-noise ratio indicates that the signal energy exceeds the error energy by more than a factor of 4×10^5, and the mean relative error of 0.28% shows that, on average, each output sample deviates from the reference by well under one percent. This residual error does not originate from any computational fault but from the deliberate use of fixed-point arithmetic. The reference, by contrast, uses 64-bit double-precision arithmetic with effectively no rounding. A small, bounded deviation between the two is therefore expected.

An SNR of 56.28 dB thus represents a high-fidelity match to the reference, sufficient for the gain-adjustment task of an audio equalizer, so the error is acceptable and the custom-instruction FFT can be considered functionally correct relative to the floating-point reference.

C. Performance Comparison

The complete audio equalizer SoC was synthesized in two configurations to quantify the hardware cost of the proposed extension. Both were synthesized for the Xilinx Zynq-7020 device in Vivado 2025.1 under an identical 20 ns (50 MHz) clock constraint, and the two builds are identical in every respect except for the custom instruction module. The difference

between the two configurations therefore isolates the cost of the proposed instructions. Table IV reports the comparison across computational performance, occupied slices, total on-chip power, and maximum operating frequency.

TABLE IV
RESOURCE AND PERFORMANCE COMPARISON

Metric	Baseline	With custom inst.	Change
FFT computation (M cycles)	17.08	5.84	65.8%
Chip area (slices)	11678	11979	2.60%
Total on-chip power (mW)	132	136	3.03%
$f_{\max}$ (MHz)	56.9	55.8	-1.93%

The results show that the 65.8% or 2.92× acceleration of the FFT computation is obtained at a small hardware cost. The number of occupied slices increases by only 2.60%, from 11,678 to 11,979. The hardware cost in power and frequency is similarly limited. Total on-chip power rises by 3.03%, from 132 mW to 136 mW. This increase is attributable entirely to the additional dynamic switching activity of the custom instruction module. The maximum operating frequency decreases by 1.93%, from 56.9 MHz to 55.8 MHz, reflecting the slightly longer critical path of the wide butterfly multipliers.

Both configurations nonetheless meet all timing constraints at the target frequency. Taken together, these results indicate that the proposed custom instructions reduce FFT computation time by 65.8% while increasing area by only 2.60% and total power by 3.03%, demonstrating that profile-driven custom instruction design delivers a favorable performance-to-cost trade-off on a minimal RISC-V core.

V. CONCLUSION

This paper presented a profile-driven design and implementation of custom RISC-V instructions for accelerating Fast Fourier Transform computation in an audio equalizer application. Four custom instructions `mshift`, `bload`, `bfly`, and `bget` collapse the four-input, two-output butterfly operation into a single hardware-accelerated sequence and minimize the multiplication operations, integrated through the standard PCPI interface without modifying the PicoRV32 core. Static instruction profiling identified the butterfly multiplication as the dominant bottleneck, motivating the choice of acceleration target. Experimental results from Icarus Verilog simulation show a 2.92× cycle count reduction for processing 1024 audio samples through a 64-point FFT-based equalizer, compared to the RV32I software-only baseline. The GNU binutils assembler was modified to recognize the custom mnemonics, enabling clean integration through inline assembly without compiler backend changes.

Future work will address three extensions. First, hardware acceleration of the bit reversal stage to capture additional cycle savings. Second, FPGA synthesis to characterize area and power overhead. Third, evaluation with larger FFT sizes ($N = 128, 256$) to assess scalability of the speedup.

REFERENCES

[1] D. Stefani and L. Turchet, "On the challenges of embedded real-time music information retrieval," in *Proc. 25th Int. Conf. Digital Audio Effects (DAFx20in22)*, Vienna, Austria, 2022.

[2] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, 1965.

[3] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[4] A. Waterman, K. Asanovic, and J. Hauser, *The RISC-V Instruction Set Manual: Volume I: Unprivileged ISA*, Document Version 20191213, RISC-V Foundation, 2019.

[5] P. Cao, H. Wang, H. Cao, Z. Wu, and Y. Liu, "A flexible dual-mode and efficient RISC-V coprocessor for on-node FFT acceleration in edge sensing," *IEEE Embedded Syst. Lett.*, 2025, doi: 10.1109/LES.2025.3634524.

[6] B. Hu, Y. Chen, and X. Zeng, "An agile instruction set extension method based on the RISC-V processor," in *Proc. 2021 IEEE 4th Int. Conf. on Electronics Technology (ICET)*, 2021, pp. 1–5, doi: 10.1109/ICET51757.2021.9450911.

[7] J. Yuan, Q. Zhao, W. Wang, X. Meng, J. Li, and Q. Li, "Audio denoising coprocessor based on RISC-V custom instruction set extension," *WSEAS Trans. Commun.*, vol. 21, pp. 189–195, Jun. 2022, doi: 10.37394/23204.2022.21.23.

[8] H. Belhadj Amor, C. Bernier, and Z. Přikryl, "A RISC-V ISA extension for ultra-low power IoT wireless signal processing," *IEEE Trans. Comput.*, vol. 71, no. 4, pp. 766–778, Apr. 2022, doi: 10.1109/TC.2021.3063027.

[9] C. Wolf, "PicoRV32—A size-optimized RISC-V CPU," GitHub repository. [Online]. Available: https://github.com/YosysHQ/picorv32

[10] E. Cui, T. Li, and Q. Wei, "RISC-V instruction set architecture extensions: A survey," *IEEE Access*, vol. 11, pp. 24696–24711, Mar. 2023, doi: 10.1109/ACCESS.2023.3246491.

[11] S. Wang, X. Wang, Z. Xu, B. Chen, C. Feng, Q. Wang, and T. T. Ye, "Optimizing CNN computation using RISC-V custom instruction sets for edge platforms," *IEEE Trans. Comput.*, vol. 73, no. 5, pp. 1371–1384, May 2024, doi: 10.1109/TC.2024.3362060.

[12] A. Garofalo, G. Tagliavini, F. Conti, L. Benini, and D. Rossi, "XpulpNN: Enabling energy efficient and flexible inference of quantized neural networks on RISC-V based IoT end nodes," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 3, pp. 1489–1505, Jul.–Sep. 2021, doi: 10.1109/TETC.2021.3072337.

[13] S. K. Mousavikia, E. Gholizadehazari, M. Mousazadeh, and S. B. Ors Yalcin, "Instruction set extension of a RiscV based SoC for driver drowsiness detection," *IEEE Access*, vol. 10, pp. 58151–58162, May 2022, doi: 10.1109/ACCESS.2022.3177743.

[14] Y. Chen, X. Wang, S. Song, L. Feng, and Z. Wang, "RISC-V custom instructions of elementary functions for IoT endpoint devices," *IEEE Trans. Comput.*, vol. 73, no. 2, pp. 523–535, Feb. 2024, doi: 10.1109/TC.2023.3336174.

[15] Communication & Multimedia LAB, NTU, "Fast Fourier Transform (FFT)." [Online]. Available: https://www.cmlab.csie.ntu.edu.tw/cml/dsp/training/coding/transform/fft.html [Accessed: May 16, 2026].

[16] V. Mahendra, "riscv-application-profiler," GitHub repository, 2023. [Online]. Available: https://github.com/mahendraVamshi/riscv-application-profiler [Accessed: May 16, 2026].